

Bases de datos gestionadas

Datos estructurados, por primera vez

Hasta ahora los datos con los que has trabajado han vivido en ficheros: un front en S3, unas imágenes en EFS. Hoy llega la primera pieza que necesita algo más estructurado — una base de datos relacional de verdad, con sus filas y sus relaciones.

Ese motor podrías instalarlo tú mismo dentro de una instancia. Hoy vas a ver la alternativa: contratarlo como servicio gestionado, y entender exactamente qué te ahorra y qué renuncias a cambio.

Instalar frente a consumir como servicio (RDS)

	Instalada por ti	Gestionada (RDS)
Tamaño de cómputo	Tipo de instancia EC2	Clase db.* — mismo catálogo, prefijo db.
Parches del motor	Los aplicas tú	AWS, en ventana de mantenimiento
Copias de seguridad	Las configuras y vigilas tú	Automáticas, con recuperación a un punto
Conmutación por error	La montas tú	Incluida si activas Multi-AZ
Acceso al sistema operativo	Total	Ninguno — ni siquiera SSH

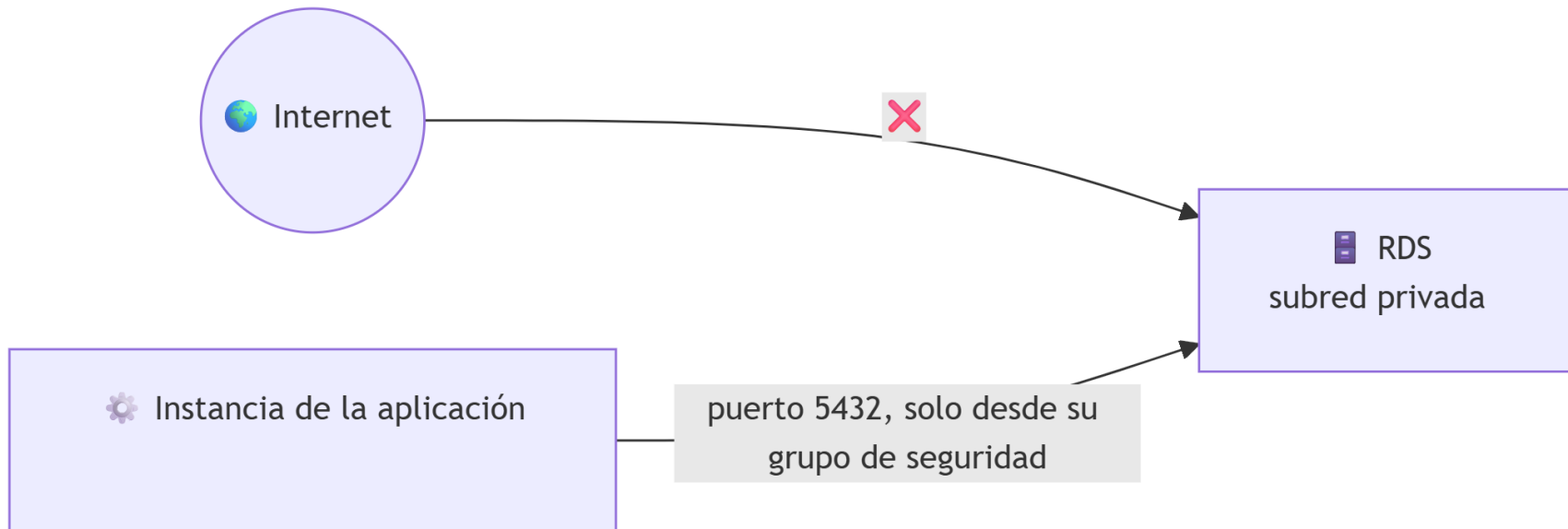
Recuperación a un punto en el tiempo

No es solo volver a la foto de la última copia diaria: RDS guarda también los cambios desde entonces, así que puedes restaurar a cualquier segundo concreto dentro de la ventana de retención — por ejemplo, un minuto antes de que alguien borrara una tabla por error.

Ninguna opción es «la buena» en abstracto — es la misma decisión de la escalera IaaS→PaaS→SaaS de la sesión 1, aplicada ahora a bases de datos.

Subred privada y grupos de seguridad de datos

La base de datos va, sin excepción, en subred privada. RDS necesita su propio grupo de seguridad, con una única regla de entrada — el puerto de la base de datos, y solo desde el grupo de seguridad de la instancia de la aplicación, nunca desde 0.0.0.0/0.



El endpoint: una dirección que no cambia para ti

Para conectarte, tu aplicación necesita un endpoint: un nombre de dirección único que sustituye a la IP de siempre — si AWS mueve la base de datos tras una conmutación por error, ese nombre sigue apuntando al sitio correcto.

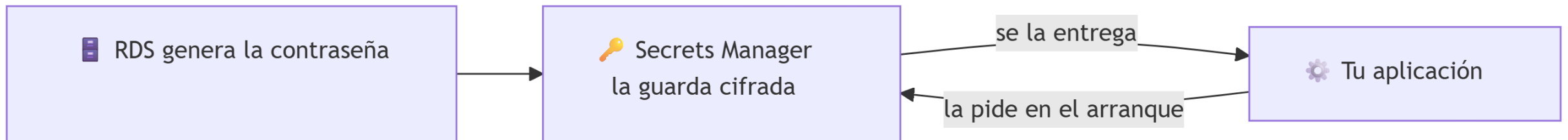


No es un escenario hipotético

Bases de datos con el puerto abierto a 0.0.0.0/0 y sin contraseña son de los hallazgos más habituales al escanear internet. «Solo desde el grupo de seguridad de la aplicación» no es una buena práctica opcional.

La contraseña no se escribe en ningún sitio

AWS Secrets Manager guarda la contraseña de forma cifrada, fuera del código, y se la entrega a quien la necesite solo en el momento de conectarse. RDS puede generarla él mismo y guardarla directamente — ni siquiera tú llegas a verla.

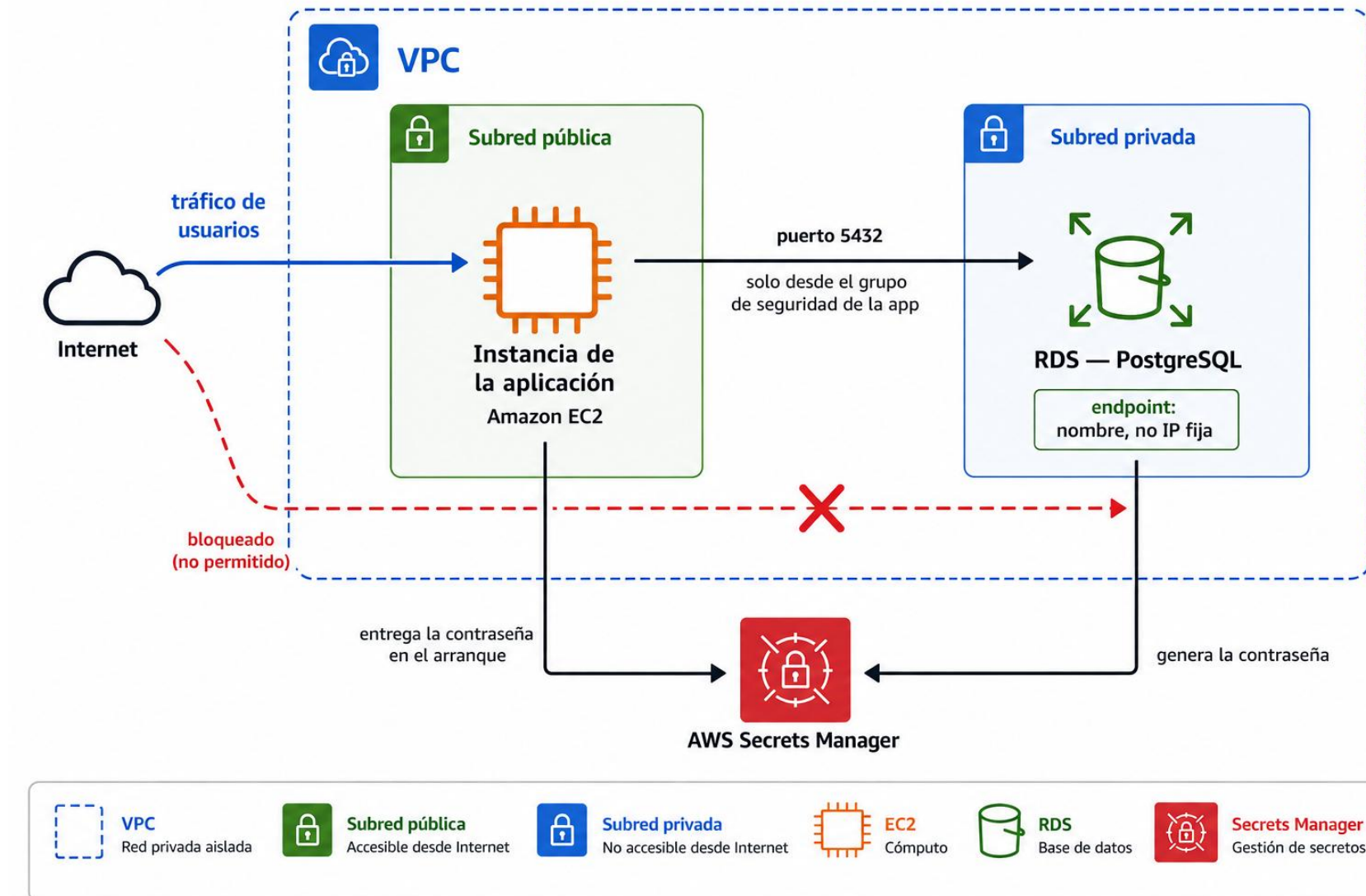


Cómo llega esto a tu aplicación, en la práctica

```
export DB_HOST=$(aws rds describe-db-instances \  
  --db-instance-identifier mi-base-de-datos \  
  --query "DBInstances[0].Endpoint.Address" --output text)  
  
aws secretsmanager get-secret-value \  
  --secret-id mi-base-de-datos-secret \  
  --query "SecretString" --output text
```

El script de arranque (user data) deja el endpoint y la contraseña como variables de entorno — nunca escritos a mano en ningún fichero de configuración.

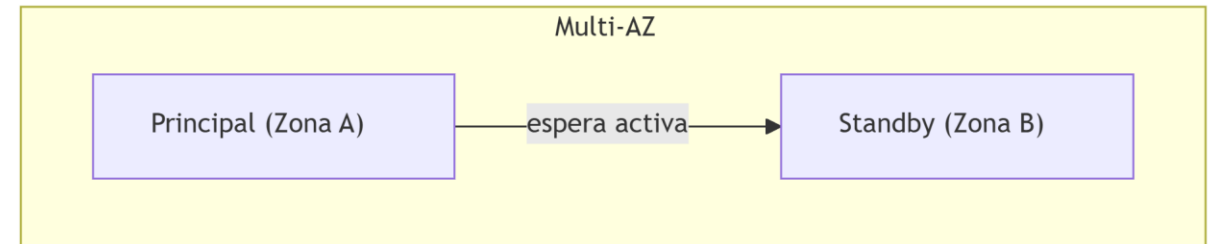
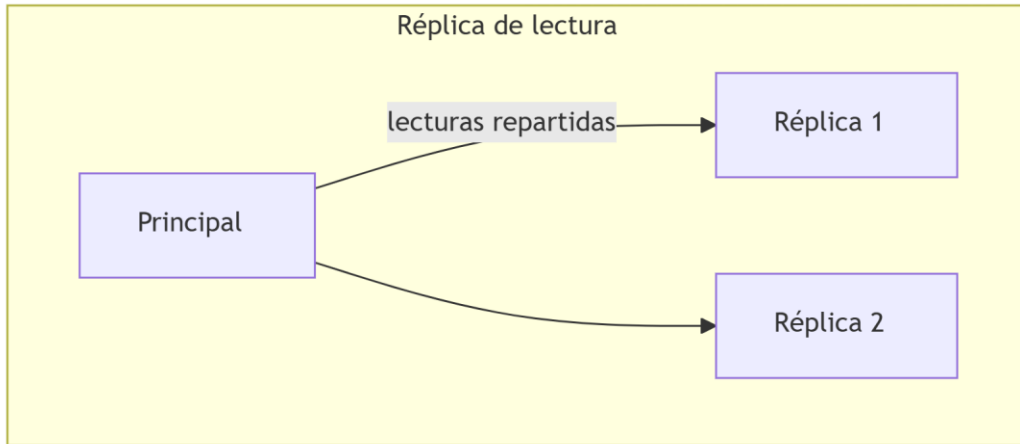
Todas las piezas, en una sola foto



Multi-AZ frente a réplica de lectura

	Multi-AZ	Réplica de lectura
Para qué existe	Alta disponibilidad	Rendimiento — repartir lecturas
¿Se puede leer/escribir?	No, pasiva hasta conmutar	Sí, solo lectura, en paralelo
Si falla la principal	AWS conmuta sola en minutos	No conmuta sola — hay que promoverla

Multi-AZ no es gratis: duplica el coste



💰 La copia en espera es una instancia completa

No es una foto ni un respaldo barato — funciona todo el tiempo aunque nunca respondas una consulta contra ella. Merece la pena cuando de verdad no puedes permitirte una caída.

Backups automáticos frente a snapshots manuales

	Backup automático	Snapshot manual
¿Quién lo crea?	RDS, solo, todos los días	Tú, cuando lo pides
¿Cuánto dura?	Lo que dure la ventana de retención	Hasta que tú lo borres
¿Sobrevive a borrar la instancia?	No	Sí

 Borrar la base de datos no borra tu snapshot manual

Si borras una instancia de RDS que solo tiene backups automáticos, esos backups se pierden con ella. Un snapshot manual sigue disponible después.

Relacional frente a NoSQL: el patrón de acceso

Modelo	Cómo se estructura	Cómo se consulta	Servicio	Cuándo encaja
Relacional	Tablas con relaciones fijas	SQL, con JOIN	RDS	Datos estructurados con relaciones claras
Clave-valor	Identificador + valor, sin estructura fija	Por la clave exacta	DynamoDB	Sesiones, carritos temporales
Documental	Documentos con estructura flexible	Comandos sobre JSON, no SQL	DocumentDB	Catálogos con atributos muy distintos

En la Actividad 3.2 vas a crear una tabla DynamoDB pequeña para comprobarlo con tus propias manos (DocumentDB se queda en razonamiento: el Learner Lab no da permisos para su clúster).

Ideas clave

- Instalar tu propia base de datos da control total; RDS asume parches, copias y conmutación por error a cambio de perder el acceso al sistema operativo.
- La base de datos va siempre en subred privada, con un grupo de seguridad que solo acepta tráfico desde la instancia de la aplicación.
- Secrets Manager guarda la contraseña cifrada y fuera del código, y puede generarla él mismo.
- Multi-AZ resuelve alta disponibilidad (copia pasiva); una réplica de lectura resuelve rendimiento (copia activa de solo lectura) — y Multi-AZ duplica el coste de cómputo.
- Los backups automáticos desaparecen si borras la instancia; un snapshot manual, no.
- Relacional (RDS), clave-valor (DynamoDB) y documental (DocumentDB) son tres modelos distintos — la elección depende de la forma de los datos y del patrón de consulta.